

FASTDIAR: FRAME-LEVEL SPEAKER ENCODER FOR STREAMING DIARIZATION

*Nikita Torgashov**

Okan Köpüklü

Department of Speech, Music and Hearing
KTH Royal Institute of Technology
Stockholm, Sweden

Microsoft
Munich, Germany

ABSTRACT

Real-time conversational agents require speaker diarization that streams and runs on a CPU. Most systems apply an utterance-level speaker encoder to short, heavily overlapping chunks, which wastes computation and leaves the model optimized for the wrong task. We instead turn a state-of-the-art speaker recognition architecture into a causal frame-level encoder that reads the stream once and emits one embedding every 80 ms from a bounded two-second window of past audio, and pair it with online clustering that gates every update on the self-similarity of the stream. Trained only by distillation from an utterance-level teacher on simulated and out-of-domain mixtures, and evaluated with one fixed set of hyperparameters, the system is the most accurate streaming diarizer on low-overlap benchmarks at sub-second latency, degrades far less than cache-based systems as the number of speakers grows, and runs five times faster than real time on a single CPU thread.

Index Terms— Speaker Diarization, Speaker Recognition, Streaming Speaker Diarization

1. INTRODUCTION

Speaker diarization answers who speaks when and is a core component of spoken conversational systems. Personalized automatic speech recognition (ASR), voice activity detection (VAD), and speech enhancement all depend on knowing which speaker is active, and a live agent must decide while the conversation unfolds, so low-latency streaming is a key requirement. GPUs are often unavailable, either because the system runs on a low-power device or because the GPU is busy with ASR, a large language model, or speech synthesis, which makes CPU diarization desirable.

Most diarization systems are built around a speaker encoder [1, 2], applied off the shelf to short, heavily overlapping chunks. That is convenient but costly: consecutive windows re-encode every sample many times over, and verification training assumes one speaker per utterance, so the encoder is never asked to forget a speaker or localize a change.

Offline diarization is dominated by cascaded pipelines combining local segmentation with clustering, such as Pyanote [3] and DiariZen [4], while end-to-end models predict activities directly with permutation-invariant training [5], attractors [6] or arrival-time ordering [7]. All of them need the whole recording before emitting labels, so none can stream.

Existing streaming systems show recurring trade-offs. Streaming Sortformer [8] keeps an arrival-order speaker cache and is accurate, but it is capped at four speakers and its 117.7M parameters are too slow on CPU. The speaker-tracing buffer [9] re-processes buffered frames at every step, BW-EDA-EEND [10] is linear in time but degrades beyond two speakers, and EEND-GLA [11] lifts the speaker-count limit without reporting a CPU budget. On the cascaded side, diart [12] attains low latency on CPU with incremental clustering over local segmentation, but is less accurate.

The encoder in front of the clustering usually stays segment-level. UIS-RNN [13] makes clustering supervised but reads d-vectors from windows with 50% overlap, and Turn-to-Diarize [14] ties output resolution to a turn detector. Two works reach frame rate: [15] distills an utterance-level teacher into a student that pools over a centred window, and LS-EEND [16, 17] learns a causal frame-wise encoder jointly with its attractor decoder. Neither gives a streaming encoder with a bounded past: the first reads the future, the second carries the whole recording in its recurrent state.

In this work we present FASTDIAR (FrAme-level speaker encoder for STreaming DIARization). Our contributions are (i) a causal frame-level speaker encoder that needs no chunking, no temporal pooling and no recomputation, and summarizes a two-second past window, (ii) an extension of incremental online clustering where an embedding-confidence gate decides which frames may shape a speaker centroid, and (iii) FASTDIAR – the most accurate streaming diarizer on low-overlap benchmarks at sub-second latency, far more robust than cache-based systems as the speaker count grows, and five times faster than real time on one CPU thread with no real diarization corpus in training. The encoder is a drop-in front end whose embeddings can feed other speaker-aware models. Code and models are available at <https://github.com/microsoft/fastdiar>.

*Work done during internship at Microsoft.

2. METHOD

2.1. Streaming speaker encoder

We take ReDimNet2 [18] as the baseline, a leading speaker recognition architecture with a low FLOPs budget. It alternates 2D convolutional blocks over a channel-frequency-time feature map with 1D blocks that operate on the same map after reshaping, and it produces a single utterance-level embedding through temporal pooling. We address three properties of this design that conflict with streaming diarization.

(1) Causal convolutions. We pad every 2D convolution only on the left of the time axis, so the output at frame t depends on frames up to t . This removes access to future context.

(2) Fully connected 1D blocks. The 1D attention blocks share temporal information across the whole utterance, giving an unbounded receptive field. That helps verification and hurts diarization, where the encoder must forget the current speaker and follow a new one. We replace them with fully connected blocks, which mix frequency bins and channels without extending the temporal receptive field.

(3) Frame-level embeddings. Temporal pooling aggregates statistics over the whole utterance and makes the past receptive field unbounded. In streaming diarization each embedding must instead rely on a limited number of past frames, so speaker changes are handled promptly and cluster boundaries stay sharp. We remove the pooling layer and project the encoder output directly to a $d = 192$ dimensional embedding at every step. Raising the time stride of the final block from 1 to 2 lowers the FLOPs count and makes each embedding summarize 80 ms of audio rather than 40 ms.

The resulting encoder has a receptive field of two seconds of past audio. It reads the stream once and emits one ℓ_2 -normalized embedding $\mathbf{e}_t$ every $\tau = 80$ ms, with no chunking and no recomputation of already seen samples. Dropping pooling bounds our past and lets an embedding forget a speaker, at a verification cost quantified in Section 4.

2.2. Online clustering

The algorithm is built around embedding confidence, the cosine similarity $\langle \mathbf{e}_t, \mathbf{e}_{t-D} \rangle$ between the current embedding and the one D frames back. High confidence means the embedding has been stable over the last D frames and the encoder has enough same-speaker context. Low confidence signals a speaker change, silence or noise. Only confident frames update the cluster model, so centroids are never degraded, while the rest are still labelled. Without it, an embedding straddling a speaker change would drag both centroids together.

Algorithm 1 summarizes the procedure. Every cluster k keeps an anchor set A_k of confident frames, represented by the ℓ_2 -normalized median $\mathbf{m}_k$ of their embeddings, which resists outliers better than the mean. A confident frame joins the closest cluster when similarity exceeds θ_{clust} , and opens a new one otherwise. Labels are not emitted immediately. Frame t

Algorithm 1: Similarity-driven online clustering

Input: embeddings $\mathbf{e}_t$, VAD v_t , look-back $D = 10$, emit delay $M = 12$; thresholds: $\theta_{\text{conf}} = 0.8$, $\theta_{\text{clust}} = 0.4$, $\theta_{\text{merge}} = 0.8$

```

1:  $\mathcal{C} \leftarrow \emptyset$ ,  $P \leftarrow \emptyset$ 
2: for  $t = 0, 1, 2, \dots$  do
3:   if  $t \geq D$  and  $\langle \mathbf{e}_t, \mathbf{e}_{t-D} \rangle \geq \theta_{\text{conf}}$  then
4:      $k \leftarrow \arg \max_{j \in \mathcal{C}} \langle \mathbf{e}_t, \mathbf{m}_j \rangle$ 
5:     if  $\mathcal{C} = \emptyset$  or  $\langle \mathbf{e}_t, \mathbf{m}_k \rangle < \theta_{\text{clust}}$  then
6:        $k \leftarrow \text{NEWCLUSTER}(\mathcal{C})$ 
7:        $A_k \leftarrow A_k \cup \{t\}$ ,  $\mathbf{m}_k \leftarrow \text{MEDIAN}(\{\mathbf{e}_i\}_{i \in A_k})$ 
8:        $R \leftarrow \{i \in P : i \leq t - M\}$ ,  $P \leftarrow (P \setminus R) \cup \{t\}$ 
9:        $\ell_i \leftarrow k$  for  $i \in R$   $\triangleright$  retroactive labels
10:      if  $k$  is new and  $R$  contains speech then  $\text{MERGE}(\mathcal{C}, \theta_{\text{merge}})$ 
11:    else  $P \leftarrow P \cup \{t\}$   $\triangleright$  unconfident, defer
12:    for  $i \in P$  with  $i \leq t - M$  do  $\triangleright$  out of context
13:       $P \leftarrow P \setminus \{i\}$ ,  $\ell_i \leftarrow \arg \max_{j \in \mathcal{C}} \langle \mathbf{e}_i, \mathbf{m}_j \rangle$   $\triangleright$  no anchor
14:    emit  $(t - M, \ell_{t-M})$  if  $v_{t-M} = 1$  else  $(t - M, \text{silence})$ 
15:  assign the frames left in  $P$  to their closest cluster

```

leaves the system when frame $t + M$ arrives, $M = 12$ frames or 960 ms fixed delay. The frames in between form a correction window. What is emitted for frame t is the cluster the stream settles on M frames later: when frame $t + M$ is confident, frame t leaves with its label, decided on 960 ms of context that frame t alone does not carry, and an online merge can revise that label on the way out, so a speaker change seen at $t + M$ sharpens the boundary before it. When frame $t + M$ is unconfident, frame t has run out of context and joins the closest median at whatever similarity, without entering an anchor set, so a guess never shapes a centroid. Without the window a low-confidence stretch holds a frame back for as long as it lasts, and the delay grows without bound exactly where fast turns, overlap and noise make it longest. An off-the-shelf VAD [19] runs on the same input stream, and every non-speech frame is emitted as silence instead of a cluster label. Whenever a new cluster receives its first speech frame, an online merge links all cluster pairs whose medians exceed θ_{merge} and fuses the connected components, repairing over-segmentation. The same merge can be optionally used over the whole recording at end of stream, as a label correction. When the stream ends, the frames still inside the window are emitted with their closest cluster. The assignment step follows incremental centroid clustering as used in online linking [20] and in diart [12], which map each embedding to the nearest centroid and open a speaker when the distance exceeds a threshold. What we add is the confidence gate. Confidence has been used in diarization before, but as a segment-level score computed after clustering, from the similarity of a segment to its assigned centroid, in order to flag likely errors for downstream consumers [21]. Ours needs no centroid and no label, arrives with the frame itself, and decides prospectively which frames may update a speaker model. The three thresholds are tuned once on the development data of Section 3.1 and then kept fixed for every experiment.

Table 1. Diarization results (DER, %). Bold marks the lowest DER in each column within an overlap group.

Model	Latency (ms)	RTF	NOTSOFAR			VoxConverse			DIHARD3			AMI	MSDWild	RAMC
			≤4 spk	≥5 spk	all	≤4 spk	≥5 spk	all	≤4 spk	≥5 spk	all	3–4 spk	2–4 spk	2 spk
<i>Consider Overlap</i>														
diart	1000	0.16	47.91	59.89	54.16	15.21	19.17	18.20	20.39	52.89	26.73	33.37	35.83	25.87
LS-EEND	1000	0.09	37.57	53.08	45.67	23.01	44.66	39.36	15.94	44.02	21.41	21.81	31.28	32.98
Streaming Sortformer	1040	1.54	16.89	37.49	27.63	6.34	24.53	20.08	15.16	41.86	20.36	30.25	23.62	25.27
FASTDIAR	960	0.19	34.59	44.25	39.63	13.76	15.49	15.07	29.42	45.62	32.57	33.36	32.61	21.61
<i>Exclude overlap</i>														
diart	1000	0.16	42.56	54.16	48.01	12.81	17.17	16.12	17.40	46.14	22.04	28.79	32.06	25.87
LS-EEND	1000	0.09	34.93	49.10	41.58	22.43	44.41	39.07	14.28	36.19	17.82	15.98	30.20	32.98
Streaming Sortformer	1040	1.54	14.48	33.45	23.39	5.41	23.55	19.15	14.08	34.25	17.34	25.56	20.81	25.27
FASTDIAR	960	0.19	19.23	24.25	21.58	10.70	12.88	12.35	24.73	32.95	26.06	22.15	24.69	21.61

3. EXPERIMENTS

3.1. Datasets

Training. VoxCeleb2 [22] contains one speaker per utterance, so multi-speaker examples are simulated on the fly. With probability 0.75 we draw crops x_1 and x_2 from two different speakers, normalize both to unit RMS, and arrange them over $T = 8$ s in one of two layouts. The first has one speaker change at $T/2$ preceded by an overlap of o ms, $\log o \sim \mathcal{N}(\mu, \sigma^2)$ truncated to $[150, 4000]$ ms and excluded from the loss. The second alternates $x_1 \rightarrow x_2 \rightarrow x_1$ with segments of at least one second. The interferer is scaled to a signal-to-interference ratio from $\mathcal{U}(-5, 5)$ dB, and the mixture is RMS-normalized so level carries no cue. We augment with MUSAN [23] and simulated room impulse responses [24] through WeSpeaker [25]. We further use the medium split of LibriHeavyMix [26], keeping unaugmented mixtures of two or three speakers in which one covers 20–90% of the audio. VoxCeleb2 is sampled 75% of the time and LibriHeavyMix 25%, with the VoxConverse and NOTSOFAR development sets used to select the checkpoint and tune the three clustering thresholds.

Benchmarks. We assess diarization on the evaluation sets of NOTSOFAR [27] (single channel), VoxConverse v0.3 [28], DIHARD3 [29], AMI [30] (IHM-MIX), MSDWild [31] (few), and RAMC [32]. They cover multiple languages, various distances and 2 to 21 speakers per recording. Streaming Sortformer is capped at four speakers, so for the three corpora crossing that boundary, we also report DER split at it. Speaker recognition is measured on VoxCeleb1-E [33].

3.2. Model and training

We distil an utterance-level teacher into a frame-level student, as in [15], but maximize cosine similarity [34] rather than minimize squared error. The teacher embeds the clean, unaugmented crop of speaker s as $\mathbf{z}_s$, and the student embeds the augmented mixture frame by frame. The objective is

$$\mathcal{L} = 1 - \frac{1}{|F|} \sum_{t \in F} \langle \mathbf{e}_t, \mathbf{z}_{s(t)} \rangle, \quad (1)$$

where $s(t)$ is the speaker active at frame t and F holds the frames with exactly one active speaker, so overlap is ignored.

The teacher is ReDimNet2-B6 trained on VoxBlink2 [35] and VoxCeleb2. Students are B2 (Small), B4 (Medium), and B6 (Large), differing only in width and FLOPs. They reuse the 2D convolution weights of the matching ReDimNet2, initialize fully connected layers from scratch, and update every weight. Training runs on one RTX 6000 Ada GPU in bf16, on 8 s segments, batch 20, 100 epochs of 2000 steps, gradients clipped at 0.5, and AdamW [36] warmed over one epoch to a peak of 1×10^{-3} then decayed linearly.

3.3. Baseline models

We compare against open-source streaming systems: Streaming Sortformer [8] v2.1 at 1040 ms latency, diart [12] at 1000 ms with recommended thresholds for DIHARD3, AMI and VoxConverse, and LS-EEND [16] at 1000 ms. All are reproduced with released code and checkpoints. LS-EEND ships one model per corpus: we use its DIHARD3 checkpoint by default, the strongest on our development data, and its AMI checkpoint for that test, so it is in-domain on DIHARD3 and AMI and out-of-domain elsewhere. To isolate the encoder we build ReDimNet2-B6-S, the unmodified ReDimNet2-B6 applied to two-second windows advanced by 80 ms, reproducing our receptive field and embedding rate and feeding the same VAD and clustering.

3.4. Evaluation protocol

We report diarization error rate (DER) with a collar of 0, with and without overlap, and split it by the number of speakers. Latency is the delay between an audio frame entering the system and its label leaving it, fixed at $M = 12$ frames, that is 960 ms, for every frame of every recording. It covers the $D = 10$ frame look-back the system fills before it can emit at all. The real-time factor (RTF) is processing time over input duration on one thread of an AMD Threadripper PRO

Table 2. VoxCeleb1-E cleaned protocol. Context and shift in ms. For the frame-level models a pair score averages over all embedding pairs. X-EER embeds the enrollment side of each trial with ReDimNet2-B6.

Model	Context	Shift	EER(%)	X-EER(%)	RTF
ReDimNet2-B6	∞	–	0.34	–	–
ReDimNet2-B6-S	2000	80	0.44	0.39	4.60
Proposed (Large)	2000	80	1.51	0.86	0.17

5995WX CPU, covering the encoder, the VAD and the clustering for FASTDIAR, and we report equal error rate (EER) for verification.

4. RESULTS AND DISCUSSION

Diarization accuracy. Results in Table 1 vary greatly under each protocol. Without overlap, FASTDIAR leads on NOTSOFAR, VoxConverse and RAMC, and trails only the in-domain LS-EEND on AMI. With overlap counted, Streaming Sortformer moves ahead of it on NOTSOFAR, DIHARD3, AMI and MSDWild, and FASTDIAR keeps VoxConverse and RAMC, the corpora with little or no overlap.

Speaker count. Streaming Sortformer is built around a cache of at most four speakers and degrades sharply beyond it: excluding overlap, its DER grows by a factor of 2.3 to 4.4 across the four-speaker boundary, against 1.2 to 1.5 for FASTDIAR, which leads the ≥ 5 column on NOTSOFAR and VoxConverse. Nothing in the encoder or the clustering is tied to a speaker count, which a streaming system rarely knows.

Baselines and efficiency. LS-EEND leads on DIHARD3 and AMI, its two in-domain corpora, and falls behind on the other four, while FASTDIAR beats diart almost everywhere. On one CPU thread FASTDIAR, diart and LS-EEND all run several times faster than real time, while Streaming Sortformer needs a GPU. FASTDIAR has the lowest algorithm latency.

Generalization. FASTDIAR trains on simulated VoxCeleb2 mixtures and LibriHeavyMix, with no real diarization corpus. Tuning the checkpoint and thresholds on their development sets leaves VoxConverse and NOTSOFAR not fully unseen, while the other four run untouched. The price shows on DIHARD3, where FASTDIAR is last under both protocols. Our simulated mixtures cover only a fraction of the scenarios it targets, and every system ahead of it has seen DIHARD3: LS-EEND trains on it, Sortformer on a mixture including its development set, and diart tunes its thresholds there.

Encoder trade-off. As Table 2 shows, the chunked model keeps verification accuracy but re-encodes every sample many times, which rules out streaming on CPU. The proposed encoder is more than 25 times faster, but it cannot pool over an utterance, so its EER is around 3 times worse. Distillation leaves the student in the teacher’s space, so the two are directly comparable. Embedding one side of every trial with

Table 3. Ablation on model scaling and offline post-processing. All models exclude overlapped speech and have 960 ms latency.

Model	#Params	RTF	VoxConverse	RAMC
ReDimNet2-B6-S	12.3M	4.85	14.79	27.97
Small	2.7M	0.05	16.49	31.59
Medium	4.6M	0.09	14.06	24.21
Large (FASTDIAR)	10.4M	0.19	12.35	21.61
+ post-processing			11.77	20.08

the offline teacher (X-EER) cuts the proposed encoder from 1.51 to 0.86 and the chunked model from 0.44 to 0.39, so one pooled side recovers more than half of the gap to the teacher. Enrollment is rarely time-critical, so a target-speaker profile, or the first centroid of a new cluster, can be built offline and then matched against streaming embeddings.

Ablation. Table 3 answers two questions. Does the frame-level encoder have to be trained, or would chunking an off-the-shelf model do? ReDimNet2-B6-S shares the receptive field and embedding rate and is the better verification model, yet Medium and Large beat it on diarization while being smaller and faster: a stronger verification model is not automatically a better diarizer. Is the result specific to one size? Accuracy improves monotonically from Small to Large, mirroring the scaling of ReDimNet2, so the recipe extrapolates rather than working by accident. Post-processing merges similar clusters at the end of the stream, outside the streaming path, showing how far offline revision can go.

Limitations. FASTDIAR’s main limitation is overlapped speech: it assigns one label per frame, so every overlapped region counts as a miss. This explains most of the gap between the halves of Table 1, and it comes from the decoding stage, not the encoder, which is trained on frames with one active speaker. In future work we plan to keep overlapped frames in the objective and predict several labels per frame, changing neither the encoder nor its inference cost.

5. CONCLUSION

We presented FASTDIAR, a streaming diarization system built on a causal frame-level speaker encoder that reads the stream once and emits an embedding every 80 ms from a two-second past window. Removing temporal pooling costs verification accuracy but buys sharp speaker-change localization and five times real-time speed on one CPU thread. With confidence-gated online clustering at a fixed 960 ms delay, it is the most accurate streaming diarizer on low-overlap benchmarks, degrades far less than cache-based systems beyond four speakers, and needs no diarization corpus in training. The encoder is the reusable part: any real-time model needing frame-level speaker identity can read the same stream, and the diarizer here is only one of its consumers.

6. REFERENCES

- [1] B. Desplanques, J. Thienpondt, and K. Demuynck, “ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification,” in *Proc. Interspeech*, 2020.
- [2] I. Yakovlev et al., “Reshape Dimensions Network for Speaker Recognition,” in *Proc. Interspeech*, 2024.
- [3] H. Bredin, “pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe,” in *Proc. Interspeech*, 2023.
- [4] J. Han et al., “Leveraging self-supervised learning for speaker diarization,” in *Proc. ICASSP*, 2025.
- [5] Y. Fujita, N. Kanda, et al., “End-to-end neural speaker diarization with self-attention,” in *Proc. ASRU*, 2019.
- [6] S. Horiguchi et al., “End-to-end speaker diarization for an unknown number of speakers with encoder-decoder based attractors,” in *Proc. Interspeech*, 2020.
- [7] T. Park, I. Medennikov, et al., “Sortformer: Seamless integration of speaker diarization and ASR by bridging timestamps and tokens,” *arXiv:2409.06656*, 2024.
- [8] I. Medennikov, T. Park, et al., “Streaming Sortformer: Speaker Cache-Based Online Speaker Diarization with Arrival-Time Ordering,” in *Proc. Interspeech*, 2025.
- [9] Y. Xue et al., “Online end-to-end neural diarization with speaker-tracing buffer,” in *Proc. SLT*, 2021.
- [10] E. Han, C. Lee, and A. Stolcke, “BW-EDA-EEND: Streaming end-to-end neural speaker diarization for a variable number of speakers,” in *Proc. ICASSP*, 2021.
- [11] S. Horiguchi, S. Watanabe, P. Garcia, Y. Takashima, and Y. Kawaguchi, “Online neural diarization of unlimited numbers of speakers using global and local attractors,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, 2023.
- [12] J. M. Coria, H. Bredin, S. Ghannay, et al., “Overlap-aware low-latency online speaker diarization based on end-to-end local segmentation,” in *Proc. ASRU*, 2021.
- [13] A. Zhang, Q. Wang, Z. Zhu, J. Paisley, et al., “Fully supervised speaker diarization,” in *Proc. ICASSP*, 2019.
- [14] W. Xia, H. Lu, Q. Wang, et al., “Turn-to-diarize: Online speaker diarization constrained by transformer transducer speaker turn detection,” in *Proc. ICASSP*, 2022.
- [15] T. Cord-Landwehr, C. Boeddeker, et al., “Frame-wise and overlap-robust speaker embeddings for meeting diarization,” in *Proc. ICASSP*, 2023.
- [16] D. Liang et al., “LS-EEND: Long-form streaming end-to-end neural diarization with online attractor extraction,” *IEEE Trans. Audio, Speech, Lang. Process.*, 2025.
- [17] D. Liang, N. Shao, and X. Li, “Frame-wise streaming end-to-end speaker diarization with non-autoregressive self-attention-based attractors,” in *Proc. ICASSP*, 2024.
- [18] I. Yakovlev and A. Okhotnikov, “ReDimNet2: Scaling Speaker Verification via Time-Pooled Dimension Reshaping,” to appear in *Proc. Interspeech*, 2026.
- [19] Silero Team, “Silero VAD: Pre-trained enterprise-grade voice activity detector,” <https://github.com/snakers4/silero-vad>, 2024.
- [20] P. A. Mansfield et al., “Links: A high-dimensional online clustering method,” *arXiv:1801.10123*, 2018.
- [21] A. Chowdhury, A. Misra, et al., “Investigating confidence estimation measures for speaker diarization,” in *Proc. Interspeech*, 2024.
- [22] J. S. Chung, A. Nagrani, et al., “VoxCeleb2: Deep speaker recognition,” in *Proc. Interspeech*, 2018.
- [23] D. Snyder, G. Chen, and D. Povey, “MUSAN: A music, speech, and noise corpus,” *arXiv:1510.08484*, 2015.
- [24] T. Ko, V. Peddinti, D. Povey, M. L. Seltzer, et al., “A study on data augmentation of reverberant speech for robust speech recognition,” in *Proc. ICASSP*, 2017.
- [25] H. Wang, C. Liang, S. Wang, Z. Chen, et al., “WeSpeaker: A research and production oriented speaker embedding learning toolkit,” in *Proc. ICASSP*, 2023.
- [26] Z. Jin, Y. Yang, M. Shi, W. Kang, X. Yang, et al., “LibriheavyMix: A 20,000-hour dataset for single-channel reverberant multi-talker speech separation, ASR and speaker diarization,” in *Proc. Interspeech*, 2024.
- [27] A. Vinnikov, A. Ivry, A. Hurvitz, et al., “NOTSOFAR-1 challenge: New datasets, baseline, and tasks for distant meeting transcription,” in *Proc. Interspeech*, 2024.
- [28] J. S. Chung et al., “Spot the conversation: Speaker diarisation in the wild,” in *Proc. Interspeech*, 2020.
- [29] N. Ryant, P. Singh, et al., “The third DIHARD diarization challenge,” in *Proc. Interspeech*, 2021.
- [30] J. Carletta, S. Ashby, et al., “The AMI meeting corpus: A pre-announcement,” in *Proc. Machine Learning for Multimodal Interaction (MLMI)*, 2005.
- [31] T. Liu et al., “MSDWild: Multi-modal speaker diarization dataset in the wild,” in *Proc. Interspeech*, 2022.
- [32] Z. Yang, Y. Chen, L. Luo, et al., “Open source MagicData-RAMC: A rich annotated Mandarin conversational speech dataset,” in *Proc. Interspeech*, 2022.
- [33] A. Nagrani et al., “VoxCeleb: A large-scale speaker identification dataset,” in *Proc. Interspeech*, 2017.
- [34] J. W. Jung et al., “Short utterance compensation in speaker verification via cosine-based teacher-student learning of speaker embeddings,” in *Proc. ASRU*, 2019.
- [35] Y. Lin, M. Cheng, et al., “VoxBlink2: A 100K+ speaker recognition corpus and the open-set speaker-identification benchmark,” in *Proc. Interspeech*, 2024.
- [36] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.